

VelociTrack: Touch Input On Uninstrumented Surfaces Using High-Speed Headset Cameras

NATHAN DEVRIO, Carnegie Mellon University, USA

VIMAL MOLLYN, Carnegie Mellon University, USA

CHRIS HARRISON, Carnegie Mellon University, USA

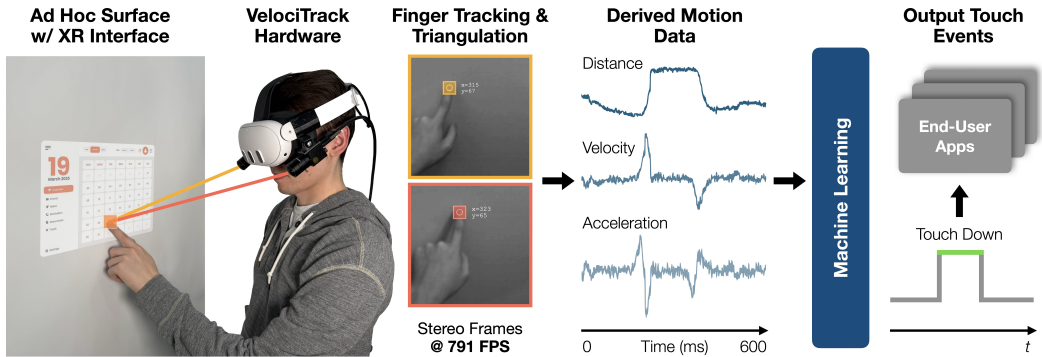


Fig. 1. VelociTrack enables touch interaction on uninstrumented physical surfaces. As a user approaches a wall-anchored XR interface (Left), two high-speed cameras (791 FPS) in the headset track and triangulate the user's fingertips, from which we derive their velocity and acceleration. Using only this motion data, our model predicts touch events, which can then be used in end-user applications.

Finger-worn IMUs have been shown to be very accurate at detecting touch events on physical surfaces, enabling input to physically-bound XR interfaces. In this work, we describe a new XR headset method for detecting finger touch events on ad hoc surfaces, without needing the user to wear anything on their fingers or hands. Instead, our method uses a pair of high-speed headset cameras to track and triangulate the position of the user's fingertips in 3D space. We take the first and second derivatives of this signal to estimate a finger's velocity and acceleration. At almost 800 Hz, our system's output is similar to a worn IMU, but with bare hands. Using this signal, we can then detect rapid and characteristic decelerations of the finger, indicative of touch.

CCS Concepts: • **Human-centered computing** → **Mixed / augmented reality; Gestural input; Touch screens**; • **Computing methodologies** → **Computer vision**.

Additional Key Words and Phrases: Computer Vision, Input Techniques, Touch Surfaces and Touch Interaction, Virtual/Augmented Reality

ACM Reference Format:

Nathan DeVrio, Vimal Mollyn, and Chris Harrison. 2026. VelociTrack: Touch Input On Uninstrumented Surfaces Using High-Speed Headset Cameras. *Proc. ACM Hum.-Comput. Interact.* 10, 5, Article MHCI8622 (August 2026), 20 pages. <https://doi.org/10.1145/3821693>

Authors' Contact Information: Nathan DeVrio, Carnegie Mellon University, Pittsburgh, PA, USA, ndevrio@cmu.edu; Vimal Mollyn, Carnegie Mellon University, Pittsburgh, PA, USA, vmollyn@cs.cmu.edu; Chris Harrison, Carnegie Mellon University, Pittsburgh, PA, USA, chris.harrison@cs.cmu.edu.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2573-0142/2026/8-ARTMHCI8622

<https://doi.org/10.1145/3821693>

1 Introduction

Appropriating surfaces in the world ad hoc for tactile touch input on virtual computing interfaces has been investigated for more than three decades [63]. With XR headsets and glasses becoming increasingly accessible and mainstream, there is a renewed effort [34, 35, 50, 54] by the HCI community to tackle this long-standing technical problem. To this literature, we contribute a new method that can be self-contained in an XR headset (or future glasses) and work across any hard surface, including transparent materials such as glass. Importantly, the user does not need to wear any special accessory on their hands, a common requirement in prior work (fingertip vibration sensors [28], finger-attached optical sensors [69], IMU rings [14, 26], wrist-worn cameras [36], etc.).

Our method uses a pair of high-speed headset cameras to track and triangulate the position of a user's fingertip in 3D space. In essence, it is a special-purpose stereo camera, producing one value: index finger depth. We take the first and second derivatives of this signal to estimate finger velocity and acceleration. At almost 800 Hz, our method's output resembles that of a finger-worn IMU—previously shown to be a useful signal [14, 51, 52]—but with bare hands. We use this virtual motion data to detect the rapid and characteristic decelerations of a finger that occur when touching a surface.

We note that unlike prior camera-based methods, operating most typically between 30-60 FPS, our technique works well for rapid touches and double taps. Not only do systems with slower cameras suffer from motion blur, but they can also entirely miss touch events due to under-sampling. In Figure 2 we show example data including a sharp corner in the depth plot when the finger decelerates upon touch down, a key feature. Fast touch events last only around 75 ms, and the telltale rapid deceleration to zero velocity lasts only ~20 ms [3]. Nyquist would imply an absolute minimum floor of 100 Hz sampling to capture this motion feature. Even still, this stream might only capture a pair of frames defining the corner, leaving little room for machine learning to integrate multiple frames to build confidence before triggering a touch event. In contrast, our hardware significantly over-samples. Not only do we believe this is more robust, but it also allowed us to run an ablation study to quantify sensor FPS vs. touch accuracy (results in Section 6.3).

In this respect, our method exhibits two advantageous properties: the remote sensing, bare-hand nature of hand tracking with a camera, and the high-framerate motion tracking reminiscent of a finger-worn IMU. After covering related work, we detail our hardware prototype and software implementation. To validate and position our approach, we conducted a user study, with results showing a touch segmentation performance of 0.94 precision and 0.94 recall. We conclude with a discussion of limitations and avenues for future work.

2 Background

At present, there is no XR device on the market that natively offers bare-hands, on-world touch input. However, these systems do provide reasonably robust 3D hand pose tracking, as well as capabilities to create 3D scans of the environment and its surfaces. Intuitively, it may seem as though one can simply combine these two information streams to enable on-world touch functionality using basic collision testing. However, inaccuracies in both the hand and world tracking compound, such that this is not actually possible at present.

We can use public benchmarks of accuracy to understand why this is the case. Meta's UmeTrack paper [17], which is presumably similar to the commercial state-of-the-art hand tracker used in the Quest line of headsets, reports a mean-per-joint-position-error (MPJPE) of 0.94 cm (i.e., roughly the thickness of a finger at its tip). We could not find any formal evaluation of 3D scene scanning accuracy for any headset, but we can visually observe its approximate nature in the visualizations of the scene mesh (see e.g., [56]). Further, Hu et al. [21] evaluated the spatial tracking accuracy of

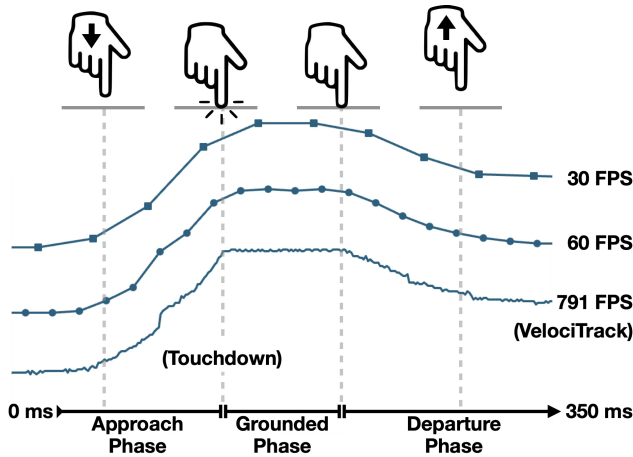


Fig. 2. Real data for an example touch event (distance from headset). Note how VelociTrack’s high framerate captures 1) the distinct corner created at the moment of impact (sudden deceleration to zero velocity), 2) the plateau while the finger stays grounded to the surface, and 3) the slower withdrawal of the finger as it pulls away from the surface. Compare this to 30 and 60 FPS data (typical sampling rates for many camera systems), which obscures most of these characteristic signals.

the Apple Vision Pro and Meta Quest 3 (both of which use visual inertial odometry), finding a mean absolute position error (APE) of 3.6 and 4.5 cm, respectively. This means any XR touch interface attached to a position in 3D space (e.g., on a wall) will likely float above or sink into a surface.

With spatial error on the order of centimeters, an intuitive collision-based approach for input sensing is not feasible. In the past, the authors have attempted to implement such functionality on multiple XR platforms and have found accuracy to be poor, requiring exaggerated finger withdrawals and considerable filter tuning to reject false positive inputs to be remotely workable. One clever approach to minimize spatial tracking errors is to calibrate the interaction plane right before use by resting a splayed hand onto a surface, as demonstrated in TriPad [7], and more recently in the experimental Meta Quest surface keyboard and trackpad [31].

However, even if future advances reduce the spatial error associated with locking UIs to surfaces, accurate touch tracking would remain a fundamental challenge. Reducing spatial error does not eliminate errors in hand keypoint estimation, which directly affect touch localization. As noted earlier, state-of-the-art hand-tracking systems currently exhibit approximately 0.94 cm of error [17]. While further improvements in hand tracking are possible, the field is relatively mature, and a fully ad hoc touch-tracking solution would likely require advances in both surface alignment accuracy and hand-tracking precision. In contrast, our approach achieves compelling touch-tracking performance using hand-tracking systems that are available today.

In summary, current systems cannot support on-world touch with their current tracking capabilities, and require additional constraints, user interactions, or sensors [7, 35, 54]. For this reason, industrial and academic research groups continue to explore methods to practically enable on-world ad hoc touch input. This is an important goal because many exciting HCI applications have been proposed assuming the capability of on-world touch, including placing virtual buttons on objects to control them [18], turning a desk into a keyboard [31, 45], or even placing user interfaces on the skin [59]. By enabling the ability to track taps independent of surface appearance, without calibration, with bare hands, and in a self-contained manner, VelociTrack actualizes these applications.

System Name	Headset-Only	Curved Surfaces	Transparent Surfaces	Sensor Location	Sensing Modality	Model Type	Performance (Acc. / F1)
FingerTouch [38]	✗	✓	✓	Fingertip	IMU ×1	Heuristic	94.8%
ActualTouch [52]	✗	✓	✓	Fingertip	IMU ×1	2D-CNN	96.6%
QwertyRing [15]	✗	✓	✓	Fingertip	IMU ×1	SVM	0.995
Gu et al. [14]	✗	✓	✓	Ring	IMU ×1	SVM	0.986
DualRing [26]	✗	✓	✓	Ring	IMU ×2	Random Forest	97.6%
MouseRing [51]	✗	✓	✓	Ring	IMU ×2	RNN	95.5%
TapID [29]	✗	✓	✓	Wrist	IMU ×2 + Tracking	Heuristic	0.996
TriPad [7]	✓	✗	✓	Headset	Hand Tracking	Heuristic	98.0%
EgoTouch [35]	✓	✓	✗	Headset	RGB Cam.	Transformer	95.6%
TapLight [54]	✓	✗	✗	Headset	Mono / RGB Cam. + Laser	Heuristic	0.953
EclipseTouch [34]	✓	✓	✗	Headset	IR Cam. / LED	Transformer	98.0%
VelociTrack (ours)	✓	✓	✓	Headset	IR Cam. ×2	1D-CNN	0.940

Table 1. Overview of key related work. VelociTrack is the only headset-based system that works on curved, complex, transparent, and specular surfaces with no other wearable instrumentation. Performance metrics are drawn from published materials as best possible; please consult individual papers for specifics.

VelociTrack could also enable new applications. For instance, an AI assistant on a pair of smart glasses could use VelociTrack’s touch output to detect when a user is touching various objects, and provide contextual information and assistance.

3 Related Systems

The value of opportunistically appropriating everyday surfaces for input has spawned considerable prior work. In this section, we provide an overview of key systems that broadly illustrate the methods researched over the past few decades. We note there is also extensive literature on touch tracking using instrumented surfaces and environments (e.g., [4, 39, 44, 66]). However, here, we solely focus on worn, mobile systems. We conclude this section with specific attention to uses of high-speed cameras for user sensing.

3.1 Ad Hoc Touch Tracking with Wearable Devices on the Hand/Arm

Intuitively, sensors placed on the arm, hand, or finger yield the greatest accuracy, since the signal is strongest there. Prior systems have leveraged this insight and explored various wearables and sensing techniques for ad hoc surface touch detection.

3.1.1 IMU. As a user taps a surface, vibrations are produced that travel up the finger, and thus a particularly robust prior approach to detecting touch events is to have users wear an IMU on the finger or wrist. These taps produce sharp and distinct spikes in the acceleration signals, resulting in very high segmentation accuracies (e.g., 97.6% in [26] and 94.8% in [38]). As such, IMU systems worn on the fingertip [38, 52] tend to perform the best, but are least practical. Practicality increases with form factors further from the finger such as a ring [14, 15, 26, 49, 51] or wristband [29], however their noisier signals [52] must be compensated with additional sensors such as a second IMU [26, 29, 51] or headset hand tracking [29].

The primary downside of IMU systems is that they require additional points of instrumentation beyond the headset. While they offer high performance, users must remember to charge and carry extra devices, which has repeatedly been found to be cumbersome and uncomfortable [10, 37]. In contrast, indirect sensing through computer vision is a more natural solution that does not inhibit natural tactile sensation, and needs no per-user calibration [8, 35]. The primary contribution of this work is using high-speed cameras to enable touch input without any specialized wearables beyond a headset or glasses, which we assume users already wear for XR.

As shown in Table 1, IMU systems typically use simple statistical features on windowed motion data (e.g., max, standard deviation, etc.) as input to heuristics on classical machine learning models. However as we discuss later in our results (Section 6), we found these models to be sensitive to the intensity of surface taps, with softer taps performing worse.

3.1.2 Acoustic. Another approach that uses similar vibroacoustic signals is acoustic microphones and transducers. Passive acoustic sensing systems, such as Skinput [20] and WhichFingers [28], used piezo films to pick up vibrations generated from a tap. Alternatively, prior work has also explored active acoustic approaches, such as SoundTrak [71] and Acustico [11], which use arrays of transducers and microphones to transmit and sense inaudible acoustic waves, which reflect differently based on touch. Acoustic approaches are favored for their simplicity, however they can have highly varying signals depending on the style of touch and can struggle in loud environments.

3.1.3 RF. The final major approach is to utilize RF signals. These systems work by transmitting, then receiving RF signals through the body or surroundings and examining fluctuations in the received signal to recognize touches. SkinTrack [73] transmitted signals from a ring to a wristband using the body as an electrical waveguide to sense touches on the skin near the wristband. ActiTouch [72] improved this idea by using hand tracking from a headset to remove the need for the ring and only use wristband electrodes. Finally Z-Ring [61], a more recent system, used a single electrode in a ring that acted as a transceiver to sense changes in bio-impedance in the hand during touches.

3.2 Ad Hoc Touch Tracking with No Additional Devices

All the previous approaches have the advantage of sensing close to the point of touch contact. However, a major drawback of all is that they require the user to wear and carry an extra device in addition to the XR headset. One possible solution, used by systems like ours, is to detect touch using only sensors inside the headset. This is typically more challenging as the signal used to detect touches can be diminished at greater distances. Once again, we break our discussion into the three most popular sensing approaches.

3.2.1 Hand Tracking. All modern XR headsets, such as the Meta Quest and Apple Vision Pro, have built-in cameras to detect and track the hands for input. For this reason, 3D hand pose has been a convenient vector from which to inform touch. Many systems, such as TouchInsight [55] and Richardson et al. [46] modeled hand motion from known behaviors, like keyboard typing, and used these models to detect touch. Alternatively, TriPad [7] had users "instrument" the surface in virtual 3D space by performing an "L" gesture, after which proximity was used to detect touches. The challenge with all such approaches is that they require the user to calibrate and pre-register each surface for input in order to account for hand tracking errors, and thus cannot support ad-hoc touch input on any surface.

3.2.2 Depth Cameras. Perhaps the most widely known prior approach for headset-only touch detection has been egocentric depth cameras. Systems such as OmniTouch [19], Imaginary Phone [16], MRTouch [67], and DIRECT [65], including Fan et al.'s successor [9], took the successful depth segmentation and flood filling ideas explored by Benko and Wilson [4] for non-worn touch detection and adapted them to the egocentric form factor. RetroDepth used stereo IR cameras to extract depth from images of the finger touching a retro-reflective surface [25]. Although the system only worked on retro-reflective surfaces, this technique allowed for easy segmentation of the finger from the surface. However, depth cameras have high power consumption, higher noise, and critically, lower speeds. In fact, we originally experimented with the RealSense D435 depth camera [23] for VelociTrack, but found its 300 FPS framerate and narrow field of view unacceptable.

3.2.3 IR & RGB Cameras. Finally the most relevant approaches to VelociTrack are those that use IR/RGB cameras. EgoTouch [35] used an egocentric RGB camera to detect touch contact and force on the user's skin (including the palm and body) by looking at patterns of skin deformation, shadow convergence and nail color change during touch events. TapLight [54] used laser speckle sensing and an IR camera to detect surfaces and when a user tapped on them. However, this approach did not function well on many common surface materials, and could fail from user head motion and multiple fingers. In our prior work, EclipseTouch [34], we used an egocentric IR camera and an IR illuminator to detect touches on ad hoc surfaces by tracking changes in the structured shadows cast by the fingers. However, EclipseTouch could not work on transparent surfaces like glass, due to the absence of shadows. VelociTrack only images the fingertip, making it agnostic to the visual appearance and physical geometry of the surface and enabling it to work on challenging surfaces like mesh and glass (see Video Figure) that other systems fail on [34, 35, 54].

3.3 High-Speed Cameras for User Sensing

Once inaccessible to consumers, cameras capable of very high framerates are becoming more commonplace. Cameras found in today's smartphones and XR headsets typically capture video at 30, 60, or 90 FPS, with slow motion capture modes running at 240 FPS or more. The Leap Motion camera runs up to 200 Hz [60], and the cameras used in high-end commercial motion capture systems can go even faster (e.g., 1000 FPS with Optitrack's Versa^x 120 [43]).

HCI researchers have employed high-speed cameras for user sensing in numerous ways. Xia et al. [64] used a 120 FPS Vicon camera mounted on a surface to track pre-touch finger motion to reduce latency on touch down prediction. Sato et al. [48] used a 160 FPS monochrome camera mounted above the hand to detect and track high-speed flick gestures. TipTrack [27] used two 158 FPS monochrome cameras, mounted above a surface, to track surface interactions with a pen. Notably, they avoided the problem we encountered of tracking the fingertip/pen at a high speed by instrumenting the tip of the pen with an IR illuminator. Finally, Niikura et al. [37] used a 145 FPS IR camera to track the in-air motion of the finger and in-air input taps, by using a bandpass filter on the fingertip Z-position. In their analysis they described how this method had limited robustness due to quick fingertip movement and noise in the depth estimation [37] and stated, "more sophisticated time-series processing will be required to improve the detection of the input action", which is exactly what VelociTrack attempts to address.

Although different from the high-speed IR cameras we use, high-speed event cameras have also been used for user sensing. Researchers have used event streams to track 3D body and hand pose both in exocentric [47, 68] and egocentric [32, 62] perspectives. In order to work, some of these systems, such as EventHands [47], also had to be trained on a large corpus of data generated synthetically from an event camera simulator. Event cameras typically work best when the motion between frames is large, thus although none of these systems explored touch detection, we estimate a high-resolution event camera would be needed to detect small finger movements.

Uniquely, VelociTrack is the first system that utilizes high-speed cameras to capture a high sampling rate estimate of fingertip position and velocity during a tap. This technique makes VelociTrack less similar to prior depth camera-based techniques and more similar to those that use worn IMUs. The high-speed motion estimates we capture are uniquely powerful signals that approximate signals obtained from an IMU worn on the fingertip, all while requiring no additional instrumentation beyond a headset.



Fig. 3. Close-up views of the VelociTrack hardware. We mounted twin HT-SUA33GM-T1V-C machine vision cameras at an angle to the sides of a Quest 3 headset. In the future, we envision high-speed cameras would be integrated directly into the headset like current inside-out tracking cameras.

4 Implementation

We now describe the details of the hardware and software components that made our proof-of-concept prototype possible. A complete overview of VelociTrack's processing pipeline is illustrated in Figure 5.

4.1 Camera Hardware

We use a pair of Huateng Vision HT-SUA33GM-T1V-C cameras [22]. These global shutter cameras offer a maximum sustained framerate of 791 FPS with a 640×480 resolution over USB 3.0. To these we attach Arducam 120° wide-angle CS lenses [2]. We purposely selected a monochrome sensor (i.e., no Bayer filter mosaic) for its improved light sensitivity, crucial at higher framerates where exposure time is limited. To boost this sensitivity further, we also removed the IR cut filter in front of the CMOS sensor to make it maximally wideband (roughly 350-1000 nm). As can be seen in figures throughout this paper, this sensitivity is sufficient in normal indoor lighting conditions.

4.2 Headset & Laptop

As a prototype headset to instrument, we used a Meta Quest 3. This, along with our two USB cameras, was tethered to a Dell G16 7630 laptop, where our software pipeline ran. The two cameras were affixed to the sides of our headset with custom 3D-printed mounts (Figures 1 and 3), with a center-to-center distance of 22 cm. The cameras are angled inwards 15° and downwards 20° so that their fields of view better overlap when the user's hands are operating in front of the user. At full framerate, each camera consumes 1.2 W for a total of 2.4 W.

4.3 Triangulation

To determine a fingertip's position in 3D space, we require two 2D camera images. Each pixel in the camera can be thought of as a 3D ray through space, with the center-most pixel in the image having a phi and theta of 0° and 0° (i.e., the normal vector). Where the two rays intersect in space is the 3D position of the target. As our cameras are on the same horizontal plane, we can efficiently approximate the depth of the tracked point with trigonometry using only the two x-positions of the fingertip.

4.4 Finger Tracking

As a proof-of-concept, we only track the tip of the index finger on the user's dominant hand, as this is a common digit for interaction. However, we note that an identical pipeline can be run for the other fingers.

Our software interleaves two different finger tracking approaches. First, we use Google's Mediapipe [12] as a heavyweight, but reliable, egocentric hand tracker. This runs at a maximum speed of

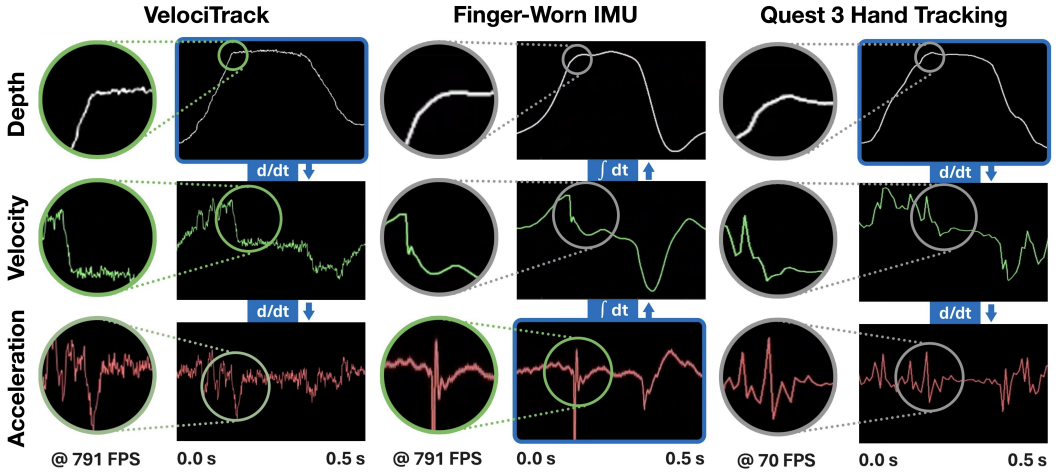


Fig. 4. Side-by-side comparison of the motion signals obtained by VelociTrack, our baseline finger-worn IMU, and Quest 3 hand tracking during a tap. VelociTrack and Quest 3 hand tracking both sense depth (top) from which we derive velocity and acceleration, while the IMU senses acceleration (bottom) which we integrate to get velocity and depth. Call-outs show the key signal used for detection. For depth this feature is a sharp corner upon touch down, for velocity it is a sharp drop then flat plateau, and for acceleration it is a large downward spike. VelociTrack captures the depth and velocity features best; for acceleration the spike is present, albeit noisier than the worn IMU. Meanwhile, the 70 FPS sampling rate of the Quest 3 hand tracking is too low to make out any features well.

~60 FPS on our laptop, and thus we cannot rely on this for high-speed tracking (i.e., at $791 \text{ FPS} \times 2$ cameras). Instead, we run MediaPipe on every 30th frame coming from our 791 FPS camera streams (i.e., ~26 Hz). For the intervening 29 camera frames, we turn to a much more efficient tracking solution: template matching.

As a template, we use a 40×40 px patch centered on the fingertip position provided by the last MediaPipe-labeled frame. On subsequent frames, we take a 50×50 px crop centered on the last known fingertip position. This small 5 px padding on all sides is sufficient to capture even fast hand motions. For example, a finger traveling at 1 m/s horizontally across the field of view, 50 cm away from our cameras, translates only 1.6 pixels per frame at 791 FPS.

Next, we scale up our fingertip template and newly cropped region by a factor of six (i.e., fingertip template = 240×240 px; camera frame crop = 300×300 px) using bilinear interpolation. We then run OpenCV's template matcher function [41] repeatedly with a small 3×3 grid search of rotations (-10° , 0° , 10°) and scales (0.9, 1.0, 1.1), and the best matching patch is selected. Due to our $6\times$ upscaling, we are able to resolve "sub-pixel" matches, which we found improves tracking accuracy and stability.

Compared to MediaPipe, our template matcher is very computationally efficient, allowing us to track the fingertip at our camera's full 791 FPS. This process is run for both cameras independently and simultaneously (i.e., our tracker is actually running at 1582 FPS). We note, however, that patch matching struggles at high hand velocities, and can drift over time as the finger changes in pose, rotation, or illumination. Using template matching as our primary algorithm, with periodic "ground truth" template updates from MediaPipe, allows us to utilize the strengths of both techniques, achieving robust and resilient tracking at very high speeds.

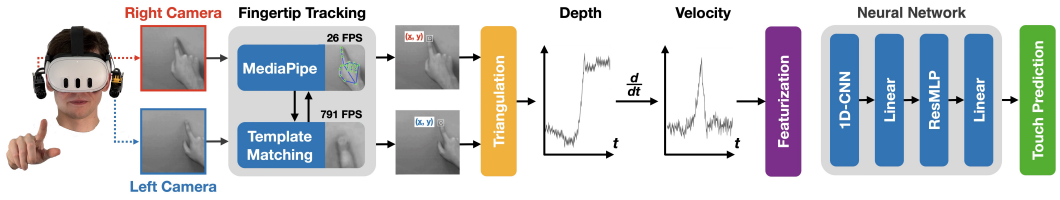


Fig. 5. The VelociTrack system pipeline. Our fingertip tracking algorithm alternates between MediaPipe and template matching to track the finger at a high speed. We then use triangulation to obtain fingertip depth and velocity. Finally, we window and featurize these streams before passing them into our neural network that produces touch predictions.

4.5 Featurization

From the depth signal produced by our fingertip tracking algorithm, we derive several features. All features are constructed as 1D windows of ~ 200 ms (160 frames), which captures the duration of a touch event [3]; we detail the windowing in Section 4.6.

The first feature is the simplest and most obvious: we differentiate consecutive frames of the depth stream to obtain a stream for fingertip velocity. We also experimented with differentiating the velocity stream once more to obtain acceleration, but the resultant stream was noisy and did not improve model performance. Of all features, velocity was the most informative, which is why it is the namesake of our system. Figure 4 also illustrates how our velocity signal best approximates the signal of an IMU.

In addition to depth and velocity, we engineered three features we found improved performance. For the first, we performed a cross-correlation of the depth window with a test signal that approximated the sharp corner and flat plateau of the depth signal on touch contact, allowing us to detect these characteristic corners (Figure 4). The remaining two features identify regions of flatness and fast change in the velocity signal. For the second, we took the velocity window and for each point calculated the variance of the 80-nearest frames, capturing where the signal is flat. For the final feature, we inverted and scaled this variance to magnify its smallest values, capturing where the signal changes fast.

4.6 Machine Learning Architecture

To design our model architecture, we first looked to models used in prior work for detecting touch from IMU signals, as even though our approach does not use an IMU, these systems also analyze motion data to solve the same problem. When an IMU is worn directly on the fingertip, the signals are strong and low noise. For this reason, statistical features (e.g., max, variance) extracted from the motion data are powerful enough to be used on their own in classical machine learning models like support vector machines (SVMs) and random forests [14, 26, 38]. However, in early experiments we found these techniques to perform poorly, perhaps due to the lack of clean, sharp spikes in our velocity data, which motivated the need for a deep learning approach.

As noted earlier, our machine learning model operates on ~ 200 ms (160 frame) windows. Our system treats the middle of the window as the "present" so functionally we predict 100 ms behind real-time. This approach of predicting past output frames was inspired by the body pose tracking literature [6, 33, 70], and offers a compromise between latency and accuracy. Features are scaled before being passed into the model but are not otherwise normalized, preserving information about absolute motion changes.

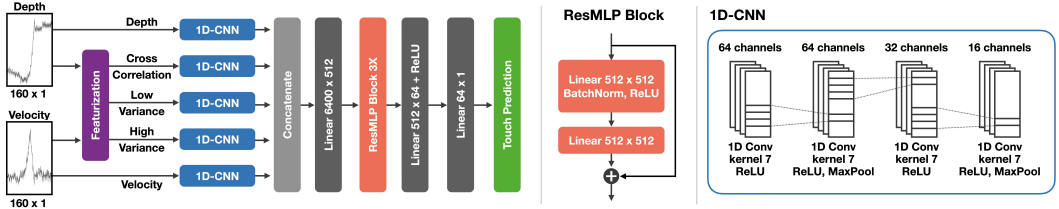


Fig. 6. The VelociTrack model architecture used to detect touch events. Our model consists of three consecutive stages including a 1D-CNN stage, a ResMLP stage, and a multi-layer perceptron stage before the output.

Our model consists of three stages (Figure 6). First each of the five features is passed through its own parallel branch of a 4-layer 1D-CNN network (similar to [29]). The output of these branches is then concatenated and passed through three ResMLP blocks [58]. Finally, we transform this output through two linear layers to predict binary touch events.

Our model has 5.1 M parameters, with a disk size of 20.5 MB. On a PC with an RTX 3080 GPU, inference time averaged 3.6 ms. Together with the built-in latency of windowing (100 ms) and featurization (15.1 ms), our system had an end-to-end latency of ~120 ms. The windowing process, while introducing latency, does not lower the throughput of touch predictions. Adding all other sources, the effective framerate of our touch prediction model is ~53.5 FPS.

4.7 Model Training Protocol

In our subsequent user study, we employ leave-one-participant-out cross validation to train and test our models. We train our model to predict touch down events, which are the most expressive parts of VelociTrack’s signal. We developed our models with PyTorch, and minimize binary cross-entropy loss using the Adam optimizer (learning rate 0.0003) for 30 epochs.

5 Evaluation

To evaluate the performance of VelociTrack we ran a user study. We recruited 10 participants (6 male, 4 female, mean age 24). Of these, 6 reported having prior experience with an XR headset. The study took 35 minutes to complete and we compensated participants \$20 for their involvement.

Each study began with a brief orientation where we explained the task and fitted the headset and finger sensors (Section 5.1). Participants then completed trials of tapping on a “surface” 60 times. Due to the features our model utilized, midair taps could potentially register as false positives. For this reason, we were interested in evaluating the ability of our system to correctly reject these false events. Each trial accordingly varied two conditions: surface orientation (horizontal/vertical) and surface type (physical/simulated in VR). This yielded four unique conditions, which participants completed once in the following order: *virtual horizontal*, *physical horizontal*, *virtual vertical*, *physical vertical*, then repeated four more times for a total of 5 repetitions $\times$ 4 conditions = 20 trials.

We did not instruct a tap cadence or withdrawal distance, but observationally the time between taps (including downtime) was about one second and the withdrawal distance ranged between 5-50 mm. This resulted in 12,000 tap events (10 participants $\times$ 60 taps $\times$ 20 trials) ~9.5 M frames of data collected (12,000 events $\times$ ~1 s $\times$ 791 FPS).

The study was conducted in an indoor office environment with only overhead fluorescent lighting. A dark gray standing desk and a whiteboard were used for the horizontal and vertical physical surfaces, respectively. For the virtual surfaces, we rendered a 0.5 $\times$ 0.5 m slab floating ~3 cm above the respective physical surface. Inside the headset, participants viewed a passthrough feed of their environment with the virtual surface rendered on top (or not) depending on the trial. We

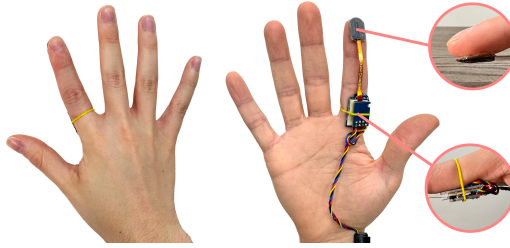


Fig. 7. Photos of our ground truth apparatus (right), which contains a fingertip SingleTact capacitive pressure sensor and ring-area IMU. Sensors were low-profile so as not to impede user touch (call-outs) and hidden on the underside of the finger so that it was not visible from headset cameras (left).

additionally played a soft sound effect to alert when touch contact was made for virtual surfaces, since they lacked tactile feedback. The ground truth sensor data was only used for the physical surface taps. For the virtual surfaces, no ground truth data was needed, since every motion frame was labeled "no tap".

5.1 Ground Truth & Baseline Data Collection

To collect ground-truth data and a reference baseline for finger taps, we instrumented the finger with two sensors (Figure 7). On the pad of the fingertip, we placed a SingleTact [53] capacitive pressure sensor and secured it using double-sided skin tape. We initially experimented with force-sensitive resistors and piezo elements, but ultimately selected the SingleTact due to its higher sensitivity, which allowed it to detect light taps that other sensors could not. The sampling rate used was the same 791 FPS as our cameras.

To generate a binary ground truth signal from pressure data, we collected an ambient pressure value at the beginning of each user evaluation. Any pressure reading that increased more than 10% above the ambient value was labeled as a 1 and otherwise a 0. We then marked 0 to 1 transitions, which were used as ground truth touch down events for our model.

In addition to the pressure sensor, users also wore a MPU9250 nine-axis IMU [57] (± 2 g range mode). As discussed in Related Work, a finger-worn IMU has been a popular and accurate way to detect ad-hoc surface touch [14, 51] due to its proximity to the tap, which is why we chose it as a reference baseline. We do not expect to beat its performance, given it senses directly from the finger, but view it as a useful standard for understanding VelociTrack's performance. We chose this particular IMU for its popularity [26, 51] and high sampling rates (>100 Hz). The MPU9250 can sample at up to 8 kHz, but signal jitter increases above 1 kHz due to I2C bus limitations, so we sampled at 791 Hz with registers set to a 218 Hz bandwidth. To match prior work [14], the IMU was placed roughly where a ring would sit. Values were collected by an ESP32 microcontroller over I2C and sent to a laptop via USB. Care was taken to hide all sensors on the underside of the finger so as not to interfere with hand and fingertip visual tracking; their exact position is shown in Figure 7.

5.2 Baseline IMU Model

For the IMU baseline, we modeled our pipeline after Gu et al. [14] which detected touch using data from an IMU on the index finger proximal phalanx. Continuous acceleration signals are grouped into 50 ms windows and five statistical features (*max*, *min*, *mean*, *skew*, and *kurtosis*) are generated for each window. These features are then used to train a support-vector machine model, which we did with the same cross validation scheme as the VelociTrack model.

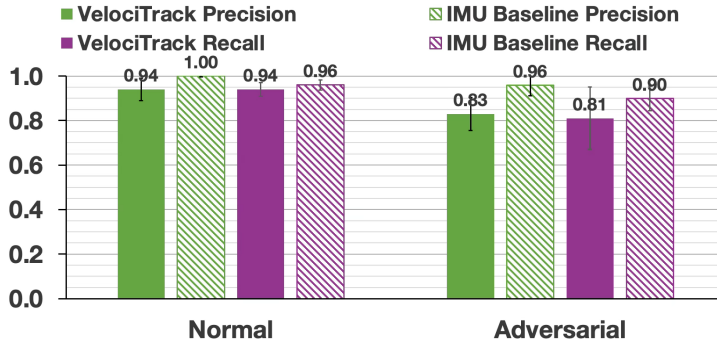


Fig. 8. Touch segmentation precision and recall for the *normal* use and *adversarial* cases. Performance for VelociTrack is compared against a finger-worn IMU baseline. Error bars are standard deviation.

6 Results & Discussion

In this section, we discuss the results of our user evaluation. The purpose of the physical taps in our study was to test performance in *normal* use, where we try to detect touches on a physical surface. This is similar to the analyses done in most prior work. To increase the ecological validity, our study also captured in-air virtual taps. This provides a set of *adversarial* data that was not performed near a surface, but has a similar motion to a tap.

Here, we break out discussion into three analyses. First, we compare performance of VelociTrack and the IMU baseline in the *normal* use case, where we trained and tested our system with on-surface taps, and the *adversarial* case, where we trained and tested our system on all data (including in-air taps). Next, we zoom in on the *normal use* case and compare performance of VelociTrack and the IMU baseline broken out for three measured tap intensities (soft, medium, hard), operating under the hypothesis that the VelociTrack method should not work as well for soft tap events. In our final analysis, we look at just VelociTrack, but perform an ablation comparing performance at our native 791 FPS framerate versus lower framerates obtained by downsampling (simulating slower, more common camera framerates). For all analyses, results are presented in precision and recall, which we calculated from continuous model predictions using the procedure outlined in Cofer et al. [5].

6.1 Analysis 1: Normal and Adversarial Use

The results for our first analysis, which compared the performance of VelociTrack against the IMU baseline for the *normal* and *adversarial* use cases, are shown in Figure 8. For the *normal* use case, our system achieved an average precision of 0.94 (SD=0.05) and recall of 0.94 (SD=0.03). These results are within 6% of the IMU baseline for precision and within 2% for recall. This means for ad hoc surface taps, VelociTrack is inside a margin of error of parity with a finger-worn IMU, one of the most common and precise techniques for worn touch detection in the past.

Meanwhile, for the *adversarial* case, our system achieved an average precision and recall of 0.83 (SD=0.07) and 0.81 (SD=0.14), which were 13.5% and 10.0% lower than the IMU baseline. VelociTrack's performance for the *adversarial* case, although decent, was lower than the *normal* case; however we did not find this surprising due to its challenging nature. Anecdotally, we saw the most false positives when users used in-air tap behaviors that mimicked an on-surface tap such as sharply halting the finger and holding it still. During smooth in-air taps and free motion, false positives were triggered at substantially lower rates.

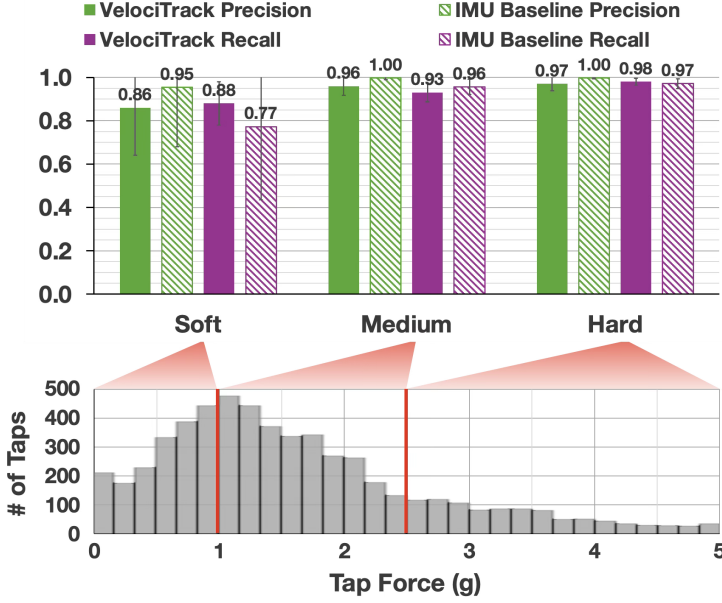


Fig. 9. Average touch precision and recall in the *normal use* case broken out for soft, medium, and hard tap force (top). Performance for VelociTrack is compared against a finger-worn IMU baseline. Error bars are standard deviation. For reference, we show the distribution of tap forces over the entire set of collected taps from our user study (bottom).

We note that most headsets build a 3D model of the environment (either for interactive use in AR, or for a "guardian" feature in VR). Thus, even today's headsets can determine when a hand is within a few centimeters of an applicable touch surface. In a commercial system, surface proximity could serve as an activation trigger for VelociTrack, and would have the benefit of filtering out the majority of adversarial taps performed in the air away from surfaces, leading to performance more closely matching our *normal use* case.

A direct comparison of VelociTrack to prior work is difficult, as reported metrics and testing conditions vary across systems. Nevertheless, in the closest comparisons available (Table 1), VelociTrack achieves similar performance while imposing no additional requirements on the user, such as extra wearables or restricted surfaces.

6.2 Analysis 2: Tap Intensity

Not all users tap in the same style—some tap harder or softer, others faster or slower. Many prior studies have not examined how tapping style affects performance [14, 26, 29, 51, 54], yet for many touch sensing approaches, especially with IMUs, softer taps can significantly dampen performance. For our second analysis, we took the *normal*-case results above and computed separate metrics for three groups based on tap intensity. We measured intensity as the peak magnitude of the IMU acceleration signal in the 100 ms window around each touch-down event, then binned taps into "soft" (<1 g), "medium" (1–2.5 g), and "hard" (>2.5 g). Full results of Analysis 2, including a histogram of all tap intensities, are shown in Figure 9.

Our goal behind this analysis was to determine if tap intensity had an effect on how well our system performed. For VelociTrack, this effect was generally confirmed to be true. VelociTrack

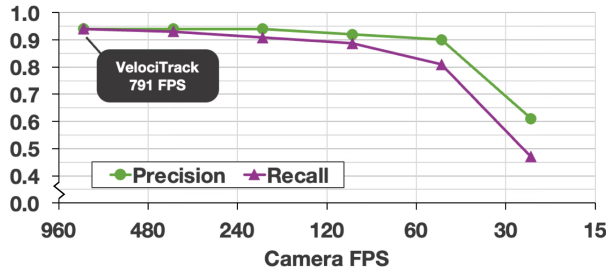


Fig. 10. Touch precision and recall results from our camera framerate ablation analysis. Camera FPS (x-axis) is shown on a log scale. Starting from VelociTrack’s native 791 FPS, we used downsampling to simulate performance of systems with lower camera framerates, all the way down to ~25 FPS.

achieved its best precision (0.97, SD=0.03) and recall (0.98, SD=0.02) for hard taps and its worst precision (0.86, SD=0.22) and recall (0.88, SD=0.1) for soft taps. This trend aligns with our features that emphasize sharp decelerations on touch down. The IMU baseline followed a similar trend, however for soft taps, its recall underperformed VelociTrack for the only time, likely due to the lack of a visible deceleration spike in very soft taps.

6.3 Analysis 3: Framerate Ablation

Finally, we conducted an ablation of VelociTrack’s performance in the *normal* use case at systematically lowered framerates. While VelociTrack’s framerate of 791 FPS is sufficient to capture surface taps, we wanted to measure the true advantage of high-speed tracking and whether more common camera framerates, such as 30 and 60 FPS, would suffice.

We took our 791 FPS depth signal and downsampled by factors of two to generate five artificial framerates all the way down to ~25 FPS. After decimating each signal, we retrained and evaluated new models at the new sampling rates.

Figure 10 plots touch detection performance against camera FPS. Overall, ablation results matched our expectations. Metrics decreased slightly, but stayed strong through the first two levels of downsampling, down to ~200 FPS, which is still a fast frame rate. Downsampling further, there was a large 8.7% drop in recall from ~100 FPS to ~50 FPS, and a precipitous drop of 32.2% in precision and 42.0% in recall from ~50 FPS to ~25 FPS.

These results indicate that if our pipeline was deployed on a commercial headset, designers could consider using cameras with framerates down to ~120 FPS *if* willing to accept small drops in performance. However, our technique would not work on slower cameras, such as the 60 FPS ones found in the Quest 3 [30].

7 Limitations & Future Work

As with any method, VelociTrack has strengths and weaknesses. We now highlight limitations and potential avenues for future work.

7.1 Accuracy

Our prototype’s accuracy shows promise, but does not meet the levels of performance expected by consumers, which is essentially 100% accuracy. As discussed in the previous section, we believe that our system’s high framerate had sufficient temporal resolution to capture the most important motion details, like an IMU does. Instead, error trials mostly stemmed from noise in our patch tracking process, including MediaPipe jitter and basic quantization issues (i.e., flickering between

pixels) as our camera resolution was only 640×480 (0.3 MP). Thus the most obvious avenues for future improvement include higher resolution cameras and a more robust hand tracker (perhaps trained on our cameras, vs. using an off-the-shelf implementation like MediaPipe). There may also be visible features we are not taking advantage of, such as fingernail blanching and contact shadows, that prior work such as PressureVision [13], EgoTouch [35] and EclipseTouch [34] have leveraged.

7.2 Tapping Style

In our evaluation we encouraged participants to perform taps naturally, which we observed led to a range of different styles and tap intensities. Nevertheless, the lab study we conducted, which is standard from prior work, could not capture the full spectrum of tapping styles that may be encountered in the wild. Future work could look into larger evaluations with more participants in order to capture a larger diversity of tapping styles.

In our analysis on tapping style, we did observe a reduced accuracy for soft touches, which showed on average a 10.8% drop in precision and 7.9% drop in recall compared to both normal and hard taps. Our system inherently relies on the rapid deceleration of a tapping finger as one of its main signals — something a lighter tap does not strongly exhibit. In a similar vein, VelociTrack can probably never support compliant surfaces, like fabrics, soft furniture and some parts of the user's skin. However, we hope to investigate these surfaces more in future work.

7.3 Lighting

We also note that VelociTrack, as tested, needs at least typical indoor lighting conditions to operate, and does not work in dark settings (though nor does any unaided camera-based hand tracker). A very recent trend has been for headsets to include built-in infrared illuminators, such that the hands are well lit in dark scenes (perfect for VelociTrack). For instance, the Meta Quest 3 did not include illuminators, but the newer and cheaper Quest 3S does (plus two monochrome cameras sensitive to infrared light to track the hands) [30]. Adding an illuminator would allow VelociTrack to work in the dark without any changes.

7.4 Hardware Cost

In terms of hardware, VelociTrack is very much a proof of concept, built to explore and validate an idea, and has not been rigorously cost optimized like a commercial product. For instance, the machine vision cameras we chose cost \$280 each. This price not only reflects retail markup, but also the cost of the ruggedized metal enclosure, power regulation, USB communication IC, etc. To get a better sense of the cost of high-speed camera sensors as they might appear in a commercially integrated headset, we must look to bulk component prices. The comparable PYTHON300 CMOS sensor (NOIP1SN0300A-QTI) made by ON Semiconductor is global shutter, 640×480 px, and 815 FPS; at order volumes over 500 the cost is around \$45 [40]. Cheaper yet is the \$10 ams OSRAM MIRA016-3QM2WB, offering 400×400 px at 740 FPS [1].

7.5 Compute

Our approach is also currently compute and memory bandwidth intensive, requiring a laptop for processing. If VelociTrack were to be integrated into a headset, we would recommend using an application-specific integrated circuit (ASIC) specialized to the task of processing image frames at a high speed. This has already been demonstrated with success in similar special-purpose computer vision hardware, such as Intel's on-board SLAM [24] and stereo depth estimation [23] running on a specialized integrated circuit paired with the camera. Compared to a general-purpose processor,

using an ASIC would significantly reduce power consumption, which is currently high for a mobile headset.

7.6 Event Cameras

Finally, we note that VelociTrack could utilize event cameras with minimal changes to its pipeline. Event cameras have the potential to unlock even higher framerates, though our ablation study results suggest this may only have modest gains in performance. We chose not to pursue this approach chiefly due to cost. The cheapest event camera on the market is the Prophesee GENX320 [42], which costs \$385 with the least expensive OpenMV Cam board. Nonetheless, if the price of these sensors continues to fall, event cameras may ultimately be a superior way to achieve high framerates in a power- and cost-efficient package.

8 Open Source

To enable other researchers to build upon our system, we have made our implementation available at <https://github.com/FIGLAB/VelociTrack>.

9 Conclusion

In this paper, we presented VelociTrack, a system for detecting touch input on uninstrumented surfaces by using high-speed cameras on a headset. Many prior systems have attempted to detect on-world touch for XR applications, however often the tap signals of interest are too small or too fast to reliably detect. Systems have also often failed on challenging surfaces like glass and mirrors that VelociTrack works on. Our novel method uses two 791 FPS cameras on the headset to take advantage of unique features of fingertip motion signals that are not present at lower frame rates. Through a user evaluation, we showed this to be the case, with VelociTrack performing significantly better than an equivalent 60 FPS system. We also showed that our system performs comparably to a finger-worn IMU during normal use. Overall, VelociTrack improves the XR experience by introducing an accurate and robust method for detecting touch on physical surfaces that could conceivably be implemented in future headsets with a small camera upgrade.

References

- [1] ams osram. 2024. ams Mira016 CMOS Image Sensor. <https://ams-osram.com/products/sensor-solutions/cmos-image-sensors/ams-mira016-cmos-image-sensor>
- [2] Arducam. 2023. Arducam CS Lens for Raspberry Pi HQ Camera, 120 Degree Ultra Wide Angle CS-Mount Lens, 3.2mm Focal Length with Manual Focus. <https://www.arducam.com/product/arducam-cs-lens-for-raspberry-pi-hq-camera-120-degree-ultra-wide-angle-cs-mount-lens-3-2mm-focal-length-with-manual-focus-ln051/>
- [3] Deanna S. Asakawa, George H. Crocker, Adam Schmaltz, and Devin L. Jindrich. 2017. Fingertip forces and completion time for index finger and thumb touchscreen gestures. *Journal of Electromyography and Kinesiology: Official Journal of the International Society of Electrophysiological Kinesiology* 34 (June 2017), 6–13. <https://doi.org/10.1016/j.jelekin.2017.02.007>
- [4] Hrvoje Benko and Andrew D. Wilson. 2009. DepthTouch: Using Depth-Sensing Camera to Enable Freehand Interactions On and Above the Interactive Surface. <https://www.semanticscholar.org/paper/DepthTouch%3A-Using-Depth-Sensing-Camera-to-Enable-On-Benko-Wilson/fdb35c7219536cd25073606bde8e700063a48863>
- [5] Savannah Cofer, Tyler N. Chen, Jackie Junrui Yang, and Sean Follmer. 2022. Detecting Touch and Grasp Gestures Using a Wrist-Worn Optical and Inertial Sensing Network. *IEEE Robotics and Automation Letters* 7, 4 (2022), 10842–10849. <https://doi.org/10.1109/LRA.2022.3191173>
- [6] Nathan DeVrio, Vimal Mollyn, and Chris Harrison. 2023. SmartPoser: Arm Pose Estimation with a Smartphone and Smartwatch Using UWB and IMU Data. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23)*. Association for Computing Machinery, New York, NY, USA, 1–11. <https://doi.org/10.1145/3586183.3606821>
- [7] Camille Dupré, Caroline Appert, Stéphanie Rey, Houssein Saidi, and Emmanuel Pietriga. 2024. TriPad: Touch Input in AR on Ordinary Surfaces with Hand Tracking Only. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*. ACM, Honolulu HI USA, 1–18. <https://doi.org/10.1145/3613904.3642323>
- [8] Ali Erol, George Bebis, Mircea Nicolescu, Richard D. Boyle, and Xander Twombly. 2007. Vision-based hand pose estimation: A review. *Computer Vision and Image Understanding* 108, 1 (Oct. 2007), 52–73. <https://doi.org/10.1016/j.cviu.2006.10.012>
- [9] Neil Xu Fan and Robert Xiao. 2022. Reducing the Latency of Touch Tracking on Ad-hoc Surfaces. *Proc. ACM Hum.-Comput. Interact.* 6, ISS (Nov. 2022), 577:489–577:499. <https://doi.org/10.1145/3567730>
- [10] Leire Francés-Morcillo, Paz Morer-Camo, María Isabel Rodríguez-Ferradas, and Aitor Cazón-Martín. 2020. Wearable Design Requirements Identification and Evaluation. *Sensors* 20, 9 (Jan. 2020), 2599. <https://doi.org/10.3390/s20092599>
- [11] Jun Gong, Aakar Gupta, and Hrvoje Benko. 2020. Acustico: Surface Tap Detection and Localization using Wrist-based Acoustic TDOA Sensing. In *Proceedings of the 33rd Annual ACM Symposium on User Interface Software and Technology (UIST '20)*. Association for Computing Machinery, New York, NY, USA, 406–419. <https://doi.org/10.1145/3379337.3415901>
- [12] Google. 2024. Hand landmarks detection guide | Google AI Edge. https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker
- [13] Patrick Grady, Chengcheng Tang, Samarth Brahmabhatt, Christopher D. Twigg, Chengde Wan, James Hays, and Charles C. Kemp. 2022. PressureVision: Estimating Hand Pressure from a Single RGB Image. <https://doi.org/10.48550/arXiv.2203.10385> arXiv:2203.10385 [cs].
- [14] Yizheng Gu, Chun Yu, Zhipeng Li, Weiqi Li, Shuchang Xu, Xiaoying Wei, and Yuanchun Shi. 2019. Accurate and Low-Latency Sensing of Touch Contact on Any Surface with Finger-Worn IMU Sensor. In *Proceedings of the 32nd Annual ACM Symposium on User Interface Software and Technology (UIST '19)*. Association for Computing Machinery, New York, NY, USA, 1059–1070. <https://doi.org/10.1145/3332165.3347947>
- [15] Yizheng Gu, Chun Yu, Zhipeng Li, Zhaocheng Li, Xiaoying Wei, and Yuanchun Shi. 2020. QwertyRing: Text Entry on Physical Surfaces Using a Ring. *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.* 4, 4 (Dec. 2020), 128:1–128:29. <https://doi.org/10.1145/3432204>
- [16] Sean Gustafson, Christian Holz, and Patrick Baudisch. 2011. Imaginary phone: learning imaginary interfaces by transferring spatial memory from a familiar device. In *Proceedings of the 24th annual ACM symposium on User interface software and technology*. ACM, Santa Barbara California USA, 283–292. <https://doi.org/10.1145/2047196.2047233>
- [17] Shangchen Han, Po-chen Wu, Yubo Zhang, Beibei Liu, Linguang Zhang, Zheng Wang, Weiguang Si, Peizhao Zhang, Yujun Cai, Tomas Hodan, Randi Cabezas, Luan Tran, Muzafer Akbay, Tsz-Ho Yu, Cem Keskin, and Robert Wang. 2022. UmeTrack: Unified multi-view end-to-end hand tracking for VR. In *SIGGRAPH Asia 2022 Conference Papers*. 1–9. <https://doi.org/10.1145/3550469.3555378> arXiv:2211.00099 [cs].
- [18] Violet Yinuo Han, Hyunsung Cho, Kiyosu Maeda, Alexandra Ion, and David Lindlbauer. 2023. BlendMR: A Computational Method to Create Ambient Mixed Reality Interfaces. *Proceedings of the ACM on Human-Computer Interaction* 7, ISS (Oct. 2023), 217–241. <https://doi.org/10.1145/3626472>

- [19] Chris Harrison, Hrvoje Benko, and Andrew D. Wilson. 2011. OmniTouch: wearable multitouch interaction everywhere. In *Proceedings of the 24th annual ACM symposium on User interface software and technology (UIST '11)*. Association for Computing Machinery, New York, NY, USA, 441–450. <https://doi.org/10.1145/2047196.2047255>
- [20] Chris Harrison, Desney Tan, and Dan Morris. 2010. Skinput: appropriating the body as an input surface. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '10)*. Association for Computing Machinery, New York, NY, USA, 453–462. <https://doi.org/10.1145/1753326.1753394>
- [21] Tianyi Hu, Tianyuan Du, Zhehan Qu, and Maria Gorlatova. 2025. XR Reality Check: What Commercial Devices Deliver for Spatial Tracking. arXiv:2508.08642 [eess.SY] <https://arxiv.org/abs/2508.08642>
- [22] HuatengVision. 2024. HT-SUA33GM-T1V-C : Huateng Vision. <https://www.huatengvision.com/product/195/>
- [23] Intel. 2023. Depth Camera D435. <https://www.intelrealsense.com/depth-camera-d435/>
- [24] Intel. 2025. Case study: Visual-Inertial Tracking. <https://www.intelrealsense.com/visual-inertial-tracking-case-study/>
- [25] David Kim, Shahram Izadi, Jakub Dostal, Christoph Rhemann, Cem Keskin, Christopher Zach, Jamie Shotton, Timothy Large, Steven Bathiche, Matthias Nießner, D. Alex Butler, Sean Fanello, and Vivek Pradeep. 2014. RetroDepth: 3D silhouette sensing for high-precision input on and above physical surfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. ACM, Toronto Ontario Canada, 1377–1386. <https://doi.org/10.1145/2556288.2557336>
- [26] Chen Liang, Chun Yu, Yue Qin, Yuntao Wang, and Yuanchun Shi. 2021. DualRing: Enabling Subtle and Expressive Hand Interaction with Dual IMU Rings. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 5, 3 (Sept. 2021), 1–27. <https://doi.org/10.1145/3478114>
- [27] Vitus Maierhöfer, Andreas Schmid, and Raphael Wimmer. 2024. TipTrack: Precise, Low-Latency, Robust Optical Pen Tracking on Arbitrary Surfaces Using an IR-Emitting Pen Tip. In *Proceedings of the Eighteenth International Conference on Tangible, Embedded, and Embodied Interaction (TEI '24)*. Association for Computing Machinery, New York, NY, USA, 1–13. <https://doi.org/10.1145/3623509.3633366>
- [28] Damien Masson, Alix Goguet, Sylvain Malacria, and Géry Casiez. 2017. WhichFingers: Identifying Fingers on Touch Surfaces and Keyboards using Vibration Sensors. In *Proceedings of the 30th Annual ACM Symposium on User Interface Software and Technology (UIST '17)*. Association for Computing Machinery, New York, NY, USA, 41–48. <https://doi.org/10.1145/3126594.3126619>
- [29] Manuel Meier, Paul Strelai, Andreas Fender, and Christian Holz. 2021. TapID: Rapid Touch Interaction in Virtual Reality using Wearable Sensing. In *2021 IEEE Virtual Reality and 3D User Interfaces (VR)*. 519–528. <https://doi.org/10.1109/VR50410.2021.00076> ISSN: 2642-5254.
- [30] Meta. 2024. Capture MR and VR Apps for Publishing. <https://developers.meta.com/horizon/resources/video-capture-mr-vr/>
- [31] Meta. 2025. How to use surface keyboard and touchpad on Meta Quest 3 and 3S | Quest Help. <https://www.meta.com/help/quest/1589938228821220/>
- [32] Christen Millerdurai, Hiroyasu Akada, Jian Wang, Diogo Luvizon, Christian Theobalt, and Vladislav Golyanik. 2024. EventEgo3D: 3D Human Motion Capture from Egocentric Event Streams. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, Seattle, WA, USA, 1186–1195. <https://doi.org/10.1109/CVPR52733.2024.00119>
- [33] Vimal Mollyn, Riku Arakawa, Mayank Goel, Chris Harrison, and Karan Ahuja. 2023. IMUPoser: Full-Body Pose Estimation using IMUs in Phones, Watches, and Earbuds. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems (CHI '23)*. Association for Computing Machinery, New York, NY, USA, 1–12. <https://doi.org/10.1145/3544548.3581392>
- [34] Vimal Mollyn, Nathan DeVrio, and Chris Harrison. 2025. EclipseTouch: Touch Segmentation on Ad Hoc Surfaces using Worn Infrared Shadow Casting. In *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology*. ACM, Busan Republic of Korea, 1–13. <https://doi.org/10.1145/3746059.3747743>
- [35] Vimal Mollyn and Chris Harrison. 2024. EgoTouch: On-Body Touch Input Using AR/VR Headset Cameras. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology (UIST '24)*. Association for Computing Machinery, New York, NY, USA, 1–11. <https://doi.org/10.1145/3654777.3676455>
- [36] Takehiro Niikura, Yoshihiro Watanabe, and Masatoshi Ishikawa. 2014. Anywhere surface touch: utilizing any surface as an input area. In *Proceedings of the 5th Augmented Human International Conference (AH '14)*. Association for Computing Machinery, New York, NY, USA, 1–8. <https://doi.org/10.1145/2582051.2582090>
- [37] Takehiro Niikura, Yoshihiro Watanabe, Takashi Komuro, and Masatoshi Ishikawa. 2014. In-Air finger motion interface for mobile devices with vibration feedback. *IEEE Transactions on Electrical and Electronic Engineering* 9, 4 (2014), 375–383. <https://doi.org/10.1002/tee.21982> eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/tee.21982>
- [38] Ju Young Oh, Ji-Hyung Park, and Jung-Min Park. 2020. FingerTouch: Touch Interaction Using a Fingernail-Mounted Sensor on a Head-Mounted Display for Augmented Reality. *IEEE Access* 8 (2020), 101192–101208. <https://doi.org/10.1109/ACCESS.2020.2997972> Conference Name: IEEE Access.

- [39] Makoto Ono, Buntarou Shizuki, and Jiro Tanaka. 2013. Touch & activate: adding interactivity to existing objects using active acoustic sensing. In *Proceedings of the 26th annual ACM symposium on User interface software and technology*. ACM, St. Andrews Scotland, United Kingdom, 31–40. <https://doi.org/10.1145/2501988.2501989>
- [40] Onsemi. 2024. PYTHON300. <https://www.onsemi.com/products/sensors/image-sensors/PYTHON300>
- [41] OpenCV. 2018. OpenCV: Object Detection. https://docs.opencv.org/4.x/df/dfb/group__imgproc__object.html#ga586ebfb0a7fb604b35a23d85391329be
- [42] OpenMV. 2024. GENX320 Event Camera Module. <https://openmv.io/products/genx320-camera-module>
- [43] OptiTrack. 2025. Versax 120 - In Depth. <http://optitrack.com/cameras/versax-120/index.html>
- [44] Siyou Pei, Pradyumna Chari, Xue Wang, Xiaoying Yang, Achuta Kadambi, and Yang Zhang. 2022. ForceSight: Non-Contact Force Sensing with Laser Speckle Imaging. In *Proceedings of the 35th Annual ACM Symposium on User Interface Software and Technology (UIST '22)*. Association for Computing Machinery, New York, NY, USA, 1–11. <https://doi.org/10.1145/3526113.3545622>
- [45] Mark Richardson, Fadi Botros, Yangyang Shi, Pinhao Guo, Bradford J Snow, Linguang Zhang, Jingming Dong, Keith Vertanen, Shugao Ma, and Robert Wang. 2024. StegoType: Surface Typing from Egocentric Cameras. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*. ACM, Pittsburgh PA USA, 1–14. <https://doi.org/10.1145/3654777.3676343>
- [46] Mark Richardson, Matt Durasoff, and Robert Wang. 2020. Decoding Surface Touch Typing from Hand-Tracking. In *Proceedings of the 33rd Annual ACM Symposium on User Interface Software and Technology*. ACM, Virtual Event USA, 686–696. <https://doi.org/10.1145/3379337.3415816>
- [47] Viktor Rudnev, Vladislav Golyanik, Jiayi Wang, Hans-Peter Seidel, Franziska Mueller, Mohamed Elgharib, and Christian Theobalt. 2021. EventHands: Real-Time Neural 3D Hand Pose Estimation from an Event Stream. In *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE, Montreal, QC, Canada, 12365–12375. <https://doi.org/10.1109/ICCV48922.2021.01216>
- [48] Toshiaki Sato, Kentaro Fukuchi, and Hideki Koike. 2006. OHAJIKI Interface: Flicking Gesture Recognition with a High-Speed Camera. In *Entertainment Computing - ICEC 2006*, Richard Harper, Matthias Rauterberg, and Marco Combetto (Eds.). Vol. 4161. Springer Berlin Heidelberg, Berlin, Heidelberg, 205–210. https://doi.org/10.1007/11872320_24 Series Title: Lecture Notes in Computer Science.
- [49] Adwait Sharma, Christina Salchow-Hömmen, Vimal Suresh Mollyn, Aditya Shekhar Nittala, Michael A. Hedderich, Marion Koelle, Thomas Seel, and Jürgen Steimle. 2023. SparseIMU: Computational Design of Sparse IMU Layouts for Sensing Fine-grained Finger Microgestures. *ACM Transactions on Computer-Human Interaction* 30, 3 (June 2023), 39:1–39:40. <https://doi.org/10.1145/3569894>
- [50] Vivian Shen, James Spann, and Chris Harrison. 2021. FarOut Touch: Extending the Range of ad hoc Touch Sensing with Depth Cameras. In *Symposium on Spatial User Interaction*. ACM, Virtual Event USA, 1–12. <https://doi.org/10.1145/3485279.3485281>
- [51] Xiyuan Shen, Chun Yu, Xutong Wang, Chen Liang, Haozhan Chen, and Yuanchun Shi. 2024. MouseRing: Always-available Touchpad Interaction with IMU Rings. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems (CHI '24)*. Association for Computing Machinery, New York, NY, USA, 1–19. <https://doi.org/10.1145/3613904.3642225>
- [52] Yilei Shi, Haimo Zhang, Kaixing Zhao, Jiashuo Cao, Mengmeng Sun, and Suranga Nanayakkara. 2020. Ready, Steady, Touch!: Sensing Physical Contact with a Finger-Mounted IMU. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 4, 2 (June 2020), 1–25. <https://doi.org/10.1145/3397309>
- [53] SingleTact. 2023. SingleTact Miniature Force Sensors | Ultra thin. <https://www.singletact.com>
- [54] Paul Strelci, Jiaxi Jiang, Juliette Rossie, and Christian Holz. 2023. Structured Light Speckle: Joint Ego-Centric Depth Estimation and Low-Latency Contact Detection via Remote Vibrometry. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. ACM, San Francisco CA USA, 1–12. <https://doi.org/10.1145/3586183.3606749>
- [55] Paul Strelci, Mark Richardson, Fadi Botros, Shugao Ma, Robert Wang, and Christian Holz. 2024. TouchInsight: Uncertainty-aware Rapid Touch and Text Input for Mixed Reality from Egocentric Vision. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*. ACM, Pittsburgh PA USA, 1–16. <https://doi.org/10.1145/3654777.3676330>
- [56] Nick Sutrich. 2023. Meta Quest 3 room scanning. <https://www.youtube.com/watch?v=xu2wgPzZNN8>. YouTube video.
- [57] TDK. 2018. MPU-9250. <https://invensense.tdk.com/products/motion-tracking/9-axis/mpu-9250/>
- [58] Hugo Touvron, Piotr Bojanowski, Mathilde Caron, Matthieu Cord, Alaaeldin El-Nouby, Edouard Grave, Gautier Izacard, Armand Joulin, Gabriel Synnaeve, Jakob Verbeek, and Hervé Jégou. 2023. ResMLP: Feedforward Networks for Image Classification With Data-Efficient Training. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45, 4 (April 2023), 5314–5321. <https://doi.org/10.1109/TPAMI.2022.3206148>

- [59] Uliana Tsimbalistaia, Caroline Berger, Hans Gellersen, and Pavel Manakhov. 2025. On-body Icons: Designing a 3D Interface for Launching Apps in Augmented Reality. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. ACM, Yokohama Japan, 1–15. <https://doi.org/10.1145/3706598.3713954>
- [60] Ultrahaptics. 2022. What is the resolution of Ultrahaptics? <https://support.ultraleap.com/hc/en-us/articles/360004368558-What-is-the-resolution-of-Ultrahaptics>
- [61] Anandghan Waghmare, Youssef Ben Taleb, Ishan Chatterjee, Arjun Narendra, and Shwetak Patel. 2023. Z-Ring: Single-Point Bio-Impedance Sensing for Gesture, Touch, Object and User Recognition. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. ACM, Hamburg Germany, 1–18. <https://doi.org/10.1145/3544548.3581422>
- [62] Luming Wang, Hao Shi, Xiaoting Yin, Kailun Yang, and Kaiwei Wang. 2025. EgoEvGesture: Gesture Recognition Based on Egocentric Event Camera. <https://doi.org/10.48550/arXiv.2503.12419> arXiv:2503.12419 [cs].
- [63] David Weimer and S Kicha Ganapathy. 1989. A synthetic visual environment with hand gesturing and voice input. *ACM SIGCHI Bulletin* 20, SI (1989), 235–240.
- [64] Haijun Xia, Ricardo Jota, Benjamin McCanny, Zhe Yu, Clifton Forlines, Karan Singh, and Daniel Wigdor. 2014. Zero-latency tapping: using hover information to predict touch locations and eliminate touchdown latency. In *Proceedings of the 27th annual ACM symposium on User interface software and technology (UIST '14)*. Association for Computing Machinery, New York, NY, USA, 205–214. <https://doi.org/10.1145/2642918.2647348>
- [65] Robert Xiao, Scott Hudson, and Chris Harrison. 2016. DIRECT: Making Touch Tracking on Ordinary Surfaces Practical with Hybrid Depth-Infrared Sensing. In *Proceedings of the 2016 ACM International Conference on Interactive Surfaces and Spaces*. ACM, Niagara Falls Ontario Canada, 85–94. <https://doi.org/10.1145/2992154.2992173>
- [66] Robert Xiao, Scott Hudson, and Chris Harrison. 2017. Supporting Responsive Cohabitation Between Virtual Interfaces and Physical Objects on Everyday Surfaces. *Proc. ACM Hum.-Comput. Interact.* 1, EICS (June 2017), 12:1–12:17. <https://doi.org/10.1145/3095814>
- [67] Robert Xiao, Julia Schwarz, Nick Throm, Andrew D. Wilson, and Hrvoje Benko. 2018. MRTouch: Adding Touch Input to Head-Mounted Mixed Reality. *IEEE Transactions on Visualization and Computer Graphics* 24, 4 (April 2018), 1653–1660. <https://doi.org/10.1109/TVCG.2018.2794222>
- [68] Lan Xu, Weipeng Xu, Vladislav Golyanik, Marc Habermann, Lu Fang, and Christian Theobalt. 2020. EventCap: Monocular 3D Capture of High-Speed Human Motions Using an Event Camera. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 4967–4977. <https://doi.org/10.1109/CVPR42600.2020.00502> ISSN: 2575-7075.
- [69] Xing-Dong Yang, Tovi Grossman, Daniel Wigdor, and George Fitzmaurice. 2012. Magic finger: always-available input through finger instrumentation. In *Proceedings of the 25th annual ACM symposium on User interface software and technology (UIST '12)*. Association for Computing Machinery, New York, NY, USA, 147–156. <https://doi.org/10.1145/2380116.2380137>
- [70] Xinyu Yi, Yuxiao Zhou, and Feng Xu. 2021. TransPose: real-time 3D human translation and pose estimation with six inertial sensors. *ACM Trans. Graph.* 40, 4 (July 2021), 86:1–86:13. <https://doi.org/10.1145/3450626.3459786>
- [71] Cheng Zhang, Qiuyue Xue, Anandghan Waghmare, Sumeet Jain, Yiming Pu, Sinan Hersek, Kent Lyons, Kenneth A. Cunefare, Omer T. Inan, and Gregory D. Abowd. 2017. SoundTrak: Continuous 3D Tracking of a Finger Using Active Acoustics. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 1, 2 (June 2017), 1–25. <https://doi.org/10.1145/3090095>
- [72] Yang Zhang, Wolf Kienzie, Yanjun Ma, Shiu S. Ng, Hrvoje Benko, and Chris Harrison. 2019. ActiTouch: Robust Touch Detection for On-Skin AR/VR Interfaces. In *Proceedings of the 32nd Annual ACM Symposium on User Interface Software and Technology (UIST '19)*. Association for Computing Machinery, New York, NY, USA, 1151–1159. <https://doi.org/10.1145/3332165.3347869>
- [73] Yang Zhang, Junhan Zhou, Gierad Laput, and Chris Harrison. 2016. SkinTrack: Using the Body as an Electrical Waveguide for Continuous Finger Tracking on the Skin. In *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*. ACM, San Jose California USA, 1491–1503. <https://doi.org/10.1145/2858036.2858082>